

VNUHCM - University of Science
Faculty of Information Technology

CS486 – Lecture 9

Functional Dependency & Normal Forms

Le Thi Nhan
ltuhan@fit.hcmus.edu.vn

Content

- **Introduction**
- Functional dependency
 - Definition
 - Inference rules
 - Keys
- Normal Form
 - 1NF
 - 2NF
 - 3NF
 - BCNF (Boyce-Codd)

Introduction

- The **goal** of relational DB design: A set of relation schemas allowing to
 - Store information without unnecessary **redundancy**
 - **Retrieve** information **easily**
- How is a bad/good DB design?
- How to evaluate the relational DB schema?

Evaluation of DB schema

- (1) Informal measures
 - Semantics of the attributes
 - Reducing the redundant values in tuples
 - Reducing the NULL values in tuples
 - Disallowing the possibility of generating spurious tuples

- (2) Normal forms (NF)
 - 1NF
 - 2NF
 - 3NF
 - BCNF (Boyce-Codd)

Semantics of attributes

■ Semantic ~ meanings

- How to interpret the attribute values stored in a tuple of the relation?
- How the attribute values in a tuple relate to one another?

■ Solutions

- Design a relational schema so that
 - The meaning of a relation's attributes can be explained easily
 - *An informal measure of how well the relation is designed*
 - It has a well-defined and unambiguous interpretation

Redundant information

- A part of information can be derived from some parts of other information

StaffBranch

staffNo	sName	position	salary	branchNo	bAddress
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London

Staff

staffNo	sName	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SG5	Susan Brand	Manager	24000	B003
SL41	Julie Lee	Assistant	9000	B005

Branch

branchNo	bAddress
B005	22 Deer Rd, London
B007	16 Argyll St, Aberdeen
B003	163 Main St, Glasgow

Source: Silberschatz et al, Database System Concepts

Reducing the redundancy

- Design of relational database schema
 - Requires to collect *good relation schemas*
- If a relation schema is *bad*
 - May lead to a risk of the redundant information
 - Be difficult in insertion, deletion, and update

Title	Year	Length	FilmType	StudioName	StarName
Star Wars	1977	124	color	Fox	Carrie Fisher
Star Wars	1977	124	color	Fox	Mark Hamill
Star Wars	1977	124	color	Fox	Harrison Ford
Mighty Ducks	1991	104	color	Disney	Emilio Esteves
Wayne's World	1992	95	color	Paramount	Dana Carvey
Wayne's World	1992	95	color	Paramount	Mike Meyers

Schema *Movies* is a good or bad design?

Good or bad design

■ Schema *Movies*

Missing out
the update

Title	Year	Length	FilmType	StudioName	StarName
Star Wars	1977	124	color	Fox	Carrie Fisher
Star Wars	1977	124	color	Fox	Mark Hamill
Star Wars	1977	124	color	Fox	Harrison Ford
Mighty Ducks	1991	104	color	Disney	Emilio Esteves
Wayne's World	1992	95	color	Paramount	Dana Carvey
Wayne's World	1992	95	color	Paramount	Mike Meyers

Insert precisely

Delete a
movie too

Reducing the redundancy

- Split *Movies* into

Movies_1

Title	Year	Length	FilmType	StudioName
Star Wars	1977	124	color	Fox
Mighty Ducks	1991	104	color	Disney
Wayne's World	1992	95	color	Paramount

~~Missing out
the update~~

~~Redundant~~

Redundant

Movies_2

Tilte	Year	StarName
Star Wars	1977	Carrie Fisher
Star Wars	1977	Mark Hamill
Star Wars	1977	Harrison Ford
Mighty Ducks	1991	Emilio Esteves
Wayne's World	1992	Dana Carvey
Wayne's World	1992	Mike Meyers

~~Delete a
movie too~~

Null value in tuples

- Many of the attributes do not apply to all tuples
 - The attribute *does not apply* to this tuple
 - The attribute value for this tuple is *unknown*
 - The value is *known but absent*
- Problems
 - Waste the storage space
 - Be difficult to do JOIN operations, COUNT or SUM aggregate operations

Generation of spurious tuples

- Joins on attributes that are neither foreign key nor primary key produce *spurious tuples*

- Example
 - Consider the relation schema
 - EMP_PRO(SSN, PNumber, Hours, EName, PName, PLocation)
 - Each tuple
 - Relates an employee to a project
 - Also includes the employee name, project name, and project location

Generation of spurious tuples

- Example: states of *EMP_PRO*

<u>Ssn</u>	<u>Pnumber</u>	Hours	Ename	Pname	Plocation
123456789	1	32.5	Smith, John B.	ProductX	Bellaire
123456789	2	7.5	Smith, John B.	ProductY	Sugarland
666884444	3	40.0	Narayan, Ramesh K.	ProductZ	Houston
453453453	1	20.0	English, Joyce A.	ProductX	Bellaire
453453453	2	20.0	English, Joyce A.	ProductY	Sugarland
333445555	2	10.0	Wong, Franklin T.	ProductY	Sugarland
333445555	3	10.0	Wong, Franklin T.	ProductZ	Houston
333445555	10	10.0	Wong, Franklin T.	Computerization	Stafford
333445555	20	10.0	Wong, Franklin T.	Reorganization	Houston
999887777	30	30.0	Zelaya, Alicia J.	Newbenefits	Stafford
999887777	10	10.0	Zelaya, Alicia J.	Computerization	Stafford
987987987	10	35.0	Jabbar, Ahmad V.	Computerization	Stafford
987987987	30	5.0	Jabbar, Ahmad V.	Newbenefits	Stafford
987654321	30	20.0	Wallace, Jennifer S.	Newbenefits	Stafford
987654321	20	15.0	Wallace, Jennifer S.	Reorganization	Houston
888665555	20	Null	Borg, James E.	Reorganization	Houston

Generation of spurious tuples

■ Split *EMP_PROJ*

EMP_LOCS

Ename	Plocation
Smith, John B.	Bellaire
Smith, John B.	Sugarland
Narayan, Ramesh K.	Houston
English, Joyce A.	Bellaire
English, Joyce A.	Sugarland
Wong, Franklin T.	Sugarland
Wong, Franklin T.	Houston
Wong, Franklin T.	Stafford
Zelaya, Alicia J.	Stafford
Jabbar, Ahmad V.	Stafford
Wallace, Jennifer S.	Stafford
Wallace, Jennifer S.	Houston
Borg, James E.	Houston

EMP_PROJ1

Ssn	Pnumber	Hours	Pname	Plocation
123456789	1	32.5	ProductX	Bellaire
123456789	2	7.5	ProductY	Sugarland
666884444	3	40.0	ProductZ	Houston
453453453	1	20.0	ProductX	Bellaire
453453453	2	20.0	ProductY	Sugarland
333445555	2	10.0	ProductY	Sugarland
333445555	3	10.0	ProductZ	Houston
333445555	10	10.0	Computerization	Stafford
333445555	20	10.0	Reorganization	Houston
999887777	30	30.0	Newbenefits	Stafford
999887777	10	10.0	Computerization	Stafford
987987987	10	35.0	Computerization	Stafford
987987987	30	5.0	Newbenefits	Stafford
987654321	30	20.0	Newbenefits	Stafford
987654321	20	15.0	Reorganization	Houston
888665555	20	NULL	Reorganization	Houston

Generation of spurious tuples

- Attempt a natural join operation

	Ssn	Pnumber	Hours	Pname	Plocation	Ename
	123456789	1	32.5	ProductX	Bellaire	Smith, John B.
*	123456789	1	32.5	ProductX	Bellaire	English, Joyce A.
	123456789	2	7.5	ProductY	Sugarland	Smith, John B.
*	123456789	2	7.5	ProductY	Sugarland	English, Joyce A.
*	123456789	2	7.5	ProductY	Sugarland	Wong, Franklin T.
	666884444	3	40.0	ProductZ	Houston	Narayan, Ramesh K.
*	666884444	3	40.0	ProductZ	Houston	Wong, Franklin T.
*	453453453	1	20.0	ProductX	Bellaire	Smith, John B.
	453453453	1	20.0	ProductX	Bellaire	English, Joyce A.
*	453453453	2	20.0	ProductY	Sugarland	Smith, John B.
	453453453	2	20.0	ProductY	Sugarland	English, Joyce A.
*	453453453	2	20.0	ProductY	Sugarland	Wong, Franklin T.
*	333445555	2	10.0	ProductY	Sugarland	Smith, John B.
*	333445555	2	10.0	ProductY	Sugarland	English, Joyce A.
	333445555	2	10.0	ProductY	Sugarland	Wong, Franklin T.
*	333445555	3	10.0	ProductZ	Houston	Narayan, Ramesh K.
	333445555	3	10.0	ProductZ	Houston	Wong, Franklin T.
	333445555	10	10.0	Computerization	Stafford	Wong, Franklin T.
*	333445555	20	10.0	Reorganization	Houston	Narayan, Ramesh K.
	333445555	20	10.0	Reorganization	Houston	Wong, Franklin T.

Generation of spurious tuples

- Cannot recover the information that was originally in EMP_PRO from EMP_LOCS and EMP_PROJ
- Spurious tuples
 - Marked by (*)
 - Additional tuples that were not in the original relation

Content

- Introduction
- **Functional dependency**
 - Definition
 - Inference rules
 - Keys
- Normal forms

Introduction

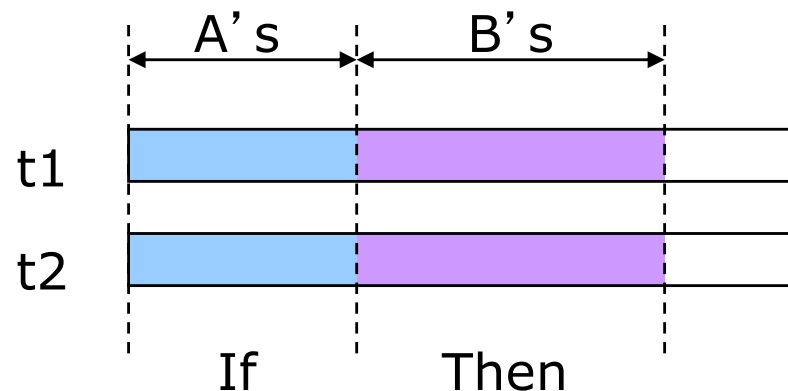
- In relational schema design
 - The most important type of constraints is a ***unique-value constraint***
 - Functional Dependency (FD)
- FD's expression is based on
 - The context of an organization
 - Rules

Using FD

- (1) To test relations to see whether they are legal under a given set of FDs
 - If a relation R is legal under a set of FDs F
 - Then **R satisfies F**
- (2) To specify constraints on the set of legal relations
 - If a schema R is constrained to satisfy a set of FDs F
 - Then **F holds on R**

Definition

- A FD on a relation R represents a relationship among sets of attributes in R
 - If two tuples of R agree on attributes $A_1, A_2, \dots, A_n$
 - Have same values in their respective components
 - Then they must agree on other attributes, $B_1, B_2, \dots, B_m$



- Denotation

$$\mathbf{A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m}$$

$A_1, A_2, \dots, A_n$ functionally determine $B_1, B_2, \dots, B_m$

Definition

- Given the relation R
 - A and B are sets of attributes, $A \neq \emptyset$, $B \neq \emptyset$
 - The instance of R, $r(R)$, satisfies the functional dependency $f: A \rightarrow B$ if

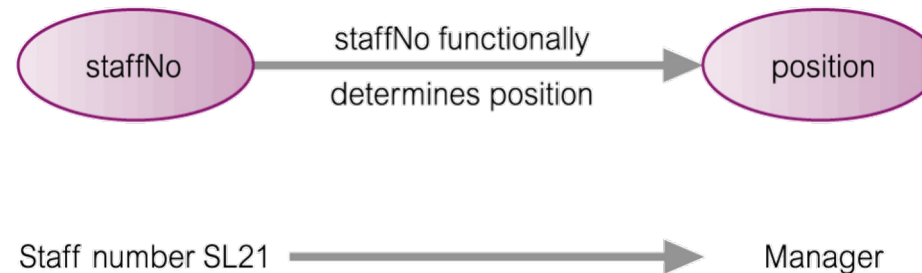
$$\forall \mathbf{t1, t2} \in \mathbf{r}, \mathbf{t1[A] = t2[A]} \Rightarrow \mathbf{t1[B] = t2[B]}$$

Each value of A defines the unique value of B

Example

Staff Branch

staffNo	sName	position	salary	branchNo	bAddress
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London

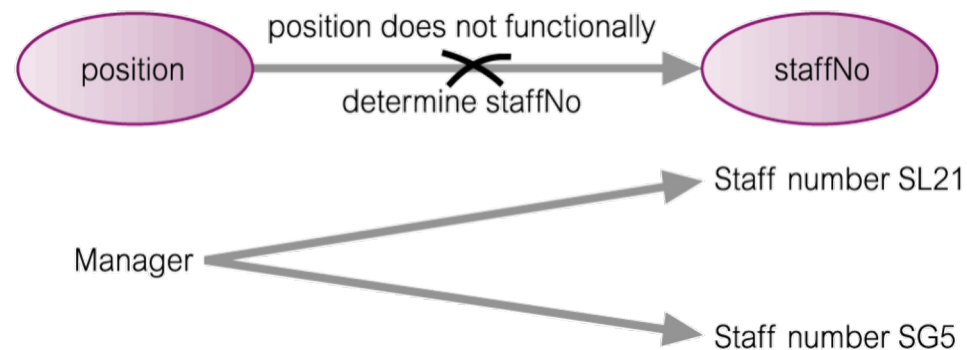


Source: Silberschatz et al, Database System Concepts

Example

Staff Branch

staffNo	sName	position	salary	branchNo	bAddress
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London



Source: Silberschatz et al, Database System Concepts

Example

■ Schema *Movies*

Title	Year	Length	FilmType	StudioName	StarName
Star Wars	1977	124	color	Fox	Carrie Fisher
Star Wars	1977	124	color	Fox	Mark Hamill
Star Wars	1977	124	color	Fox	Harrison Ford
Mighty Ducks	1991	104	color	Disney	Emilio Esteves
Wayne's World	1992	95	color	Paramount	Dana Carvey
Wayne's World	1992	95	color	Paramount	Mike Meyers

■ What are FDs that we can assert?

Example

Title, Year $\rightarrow$ Length

Title, Year $\rightarrow$ FilmType

Title, Year $\rightarrow$ StudioName

Title, Year $\rightarrow$ ~~StarName~~

Is not a FD

Notice

- Consider an instance of *Movies*, r1

Title	Year	Length	FilmType	StudioName	StarName
Star Wars	1977	124	color	Fox	Carrie Fisher
Star Wars	1977	124	color	Fox	Mark Hamill
Star Wars	1977	124	color	Fox	Harrison Ford
Mighty Ducks	1991	104	color	Disney	Emilio Esteves
Wayne's World	1992	95	color	Paramount	Dana Carvey
Wayne's World	1992	95	color	Paramount	Mike Meyers

Title → FilmType

Notice

- Consider another instance of *Movies*, r2

Title	Year	Length	FilmType	StudioName	StarName
Star Wars	1977	124	color	Fox	Carrie Fisher
Star Wars	1977	124	color	Fox	Mark Hamill
Star Wars	1977	124	color	Fox	Harrison Ford
Mighty Ducks	1991	104	color	Disney	Emilio Esteves
Kingkong	1993	120	color	Paramount	Fay Wray
Kingkong	1993	120	black/white	Paramount	Robert Amstrong

Title → ~~FilmType~~

FDs must be defined on the relation schema

Need to satisfy all instances of the relation

Inference from FDs

- Given F , the set of FDs, is defined on relation R
 - Whether R also satisfies other FDs or not?
 - If another FD f satisfies all instances of R
 - Then f is ***inferred*** from the FDs in F
- F is specified obviously by designers
- However
 - Numerous other FDs hold in all legal relation instances
- The ability of deduction
 - Aim to discover additional FDs

Example

- Schema $R(A, B, C, G, H, I)$
- And the set of FDs F is specified on R as follows

$$F = \left\{ \begin{array}{l} f1: A \rightarrow B \\ f2: A \rightarrow C \\ f3: CG \rightarrow H \\ f4: CG \rightarrow I \\ f5: B \rightarrow H \end{array} \right\}$$

- $f6$ is an inferred FD from F

$$f6: A \rightarrow H$$

Inference rules

- Armstrong's axioms

- Reflexive

$$(FD1) \quad \forall Y \subseteq X, X \rightarrow Y$$

- Augmentation

$$(FD2) \quad \begin{array}{l} \text{If } X \rightarrow Y \text{ and } Z \\ \text{Then } X, Z \rightarrow Y, Z \end{array}$$

- Transitive

$$(FD3) \quad \begin{array}{l} \text{If } X \rightarrow Y \text{ and } Y \rightarrow Z \\ \text{Then } X \rightarrow Z \end{array}$$

Inference rules

- Pseudo transitive

(FD4)

If $X \rightarrow Y$ and $Y, W \rightarrow Z$

Then $X, W \rightarrow Z$

- Combining

(FD5)

If $X \rightarrow Y$ and $X \rightarrow Z$

Then $X \rightarrow Y, Z$

- Splitting

(FD6)

If $X \rightarrow Y$ and $Z \subseteq Y$

Then $X \rightarrow Z$

Discussion

- In reality

- It is impossible to specify all possible FDs

- Example

$f1: \text{DNumber} \rightarrow \text{MgrSSN}$

$f2: \text{MgrSSN} \rightarrow \text{MgrPhone}$

$f3: \text{DNumber} \rightarrow \text{MgrPhone}$

- $f3$ is an inferred FD
- $f3$ does not need to be stated explicitly in addition to the two given FDs

Closure of attributes

- The closure set of attributes
- Given F , a set of FDs, is specified on relation R
- And $\{A1, A2, \dots, An\}$ is a set of attributes of R
- The closure of $\{A1, A2, \dots, An\}$ under the FDs in F is the set of attributes B
 - Every relation that satisfies all the FDs in F also satisfies $A1, A2, \dots, An \rightarrow B$
- Denotation

$$\{A1, A2, \dots, An\}^+$$

Example

- $R(A, B, C, D, E, F)$
- $F = \{f1:AB \rightarrow C, f2:BC \rightarrow AD, f3:D \rightarrow E, f4:CF \rightarrow B\}$
- The closure of $\{A,B\}$?
- $X = \{A,B\}$
- $f1 : X=\{A,B,C\}$
- $f2 : X=\{A,B,C,D\}$
- $f3 : X=\{A,B,C,D,E\}$
- Do not use $f4$, because its left side is not contained in X
- $\{A,B\}^+ = \{A,B,C,D,E\}$

Example

- Schema $R(A, B, C, G, H, I)$
- And the set of FDs F is specified on R as follows

$$F = \left\{ \begin{array}{l} f1: A \rightarrow B \\ f2: A \rightarrow C \\ f3: CG \rightarrow H \\ f4: CG \rightarrow I \\ f5: B \rightarrow H \end{array} \right\}$$

- $f6: A \rightarrow H$ is an inferred FD from F
- Because $H \in \{A\}^+$

Closure of a set of FDs

- The closing set of functional dependencies
- Given F , the set of FDs, is specified on relation R
- The set of all dependencies that include
 - F as well as
 - All dependencies that can be inferred from Fis called *the closure of F*
- Denotation: F^+

$$F \subseteq F^+$$

Properties

■ Equivalent

- Two sets of FDs $F1$ and $F2$ are equivalent

$$F1 \equiv F2 \Leftrightarrow F1^+ = F2^+$$

All instances of the
relation satisfy $F1$

$\equiv$

All instances of the
relation satisfy $F2$

■ Lemma

$$F1 \equiv F2 \Leftrightarrow \begin{array}{l} F1 \text{ is inferred from } F2 \text{ and} \\ F2 \text{ is inferred from } F1 \end{array}$$

Example

- $R(A, B, C, D, E)$
- $F1 = \{A \rightarrow BC, A \rightarrow D, CD \rightarrow E\}$
- $F2 = \{A \rightarrow BCE, A \rightarrow ABD, CD \rightarrow E\}$
- $F1 \equiv F2 ?$

- Proof
 - $F1$ is inferred from $F2$
 - $F2$ is inferred from $F1$

 - If $(\exists f \in F1) \wedge (\nexists f \in F2)$ then $F1 \not\equiv F2$

Example

- $\{A \rightarrow BC, A \rightarrow D, CD \rightarrow E\} \stackrel{?}{\rightarrow} \{A \rightarrow BCE, A \rightarrow ABD, CD \rightarrow E\}$
 - Obviously $F1 \subseteq F2$
- $\{A \rightarrow BCE, A \rightarrow ABD, CD \rightarrow E\} \stackrel{?}{\rightarrow} \{A \rightarrow BC, A \rightarrow D, CD \rightarrow E\}$
 - Consider $A \rightarrow E$ in $F2$
 - Whether $A \rightarrow E$ is in $F1$ or not?

Example

- $R(A, B, C, D, E)$
- $F1 = \{ A \rightarrow BC, A \rightarrow D, CD \rightarrow E \}$
- $F2 = \{ A \rightarrow BCDE \}$
- $F1 \equiv F2 ?$

- Proof
 - $F1$ is inferred from $F2$
 - $F2$ is inferred from $F1$

Example

- $\{A \rightarrow BC, A \rightarrow D, CD \rightarrow E\} \stackrel{?}{\rightarrow} \{A \rightarrow BCDE\}$
 - $A \rightarrow E$ is in F2

- $\{A \rightarrow BCDE\} \rightarrow \{A \rightarrow BC, A \rightarrow D, CD \rightarrow E\}$
 - Consider $CD \stackrel{?}{\rightarrow} E$ does not belong to F2
 - F1 is not inferred from F2

Concepts

- (1) Trivial

$X \rightarrow Y$ is trivial **if** $Y \subseteq X$

- Example

$A \rightarrow A$

$AB \rightarrow B$

$ABC \rightarrow BC$

Concepts

- (2) Complete

Examine $X \rightarrow Y$

If $\nexists X' \subset X$ so that

$$F \equiv F - \{X \rightarrow Y\} \cup \{X' \rightarrow Y\}$$

Then Y depends on X completely

Y is functional dependent on X and
not functional dependent on any subsets of X

Concepts

- (2) Complete
- Example
 - $R(A, B, C, D, E, I)$
 - $F = \{ A \rightarrow BCD, BCD \rightarrow E, CD \rightarrow EI \}$
 - $BCD \rightarrow E$ is a complete FD ???

Concepts

■ (3) Excessive

$X \rightarrow Y$ is excessive **if** $F \equiv F - \{X \rightarrow Y\}$

■ Example

- $R(A, B, C, D)$
- $F = \{ A \rightarrow B, B \rightarrow A, B \rightarrow C, A \rightarrow C, C \rightarrow A \}$
- $B \rightarrow A$ is an excessive FD?

Finding a good F

1. $G \leftarrow F$
2. Replace each FD $X \rightarrow \{A_1, A_2, \dots, A_n\}$ in G by n FDs $X \rightarrow A_1, X \rightarrow A_2, \dots, X \rightarrow A_n$
3. For each FD $X \rightarrow A$ in G
for each attribute B that is an element of X
If $((G - \{X \rightarrow A\}) \cup \{(X - \{B\}) \rightarrow A\})$ is equivalent to G ,
then replace $X \rightarrow A$ with $(X - \{B\}) \rightarrow A$ in G
4. For each remaining FD $X \rightarrow A$ in G
If $(G - \{X \rightarrow A\})$ is equivalent to G ,
then remove $X \rightarrow A$ from G

Example

- Q (LMNPRS)
- $F = \{M \rightarrow N, MR \rightarrow N, PN \rightarrow LR, L \rightarrow S, S \rightarrow R\}$

- $M \rightarrow N$ (1)
- $MR \rightarrow N$ (2) not complete FD, replace $M \rightarrow N$
- $PN \rightarrow L$ (3)
- $PN \rightarrow R$ (4) excessive FD, remove
- $L \rightarrow S$ (5)
- $S \rightarrow R$ (6)

- $F = \{M \rightarrow N, PN \rightarrow L, L \rightarrow S, S \rightarrow R\}$

In brief

- FD is a special constraint: need to implement it
- But, trivial, incomplete, and excessive FDs: should not implement
- E.g.
 - $Q(A, B, C, D) \ \& \ F = \{f1: AB \rightarrow C, f2: BC \rightarrow D\}$
 - We also have $f3: AB \rightarrow D$
 - If we implement all FDs, it is costly
 - Therefore, find a good F to ensure that all constraints are satisfied and implement at less cost

Concept of keys

■ Keys

- A set of attributes specify the uniqueness of each tuple in the relation
- Values of tuples in a relation are different at key attributes

■ Categories

- Super key
- Key
- Primary key

Example

R	A	B	C	D
	x	1	10	a
	x	2	20	a
	y	1	40	b
	y	1	40	c
	z	1	50	d

- Subsets of the set of attributes
 - ABCD
 - ABC, ABD, ACD, BCD
 - AB, AC, AD, BC, BD, CD
 - A, B, C, D
- Super keys
 - ABCD, ABD, ACD, BCD, BD, CD
- Keys
 - BD, CD

Concept of keys

■ Definition

- Given the relation R
- And F , a set of FDs, is specified on R
- $K \subseteq R^+$ is a key of R if
 - (i) $(f: K \rightarrow R^+) \in F^+ \quad (K^+ = R^+)$
 - (ii) $\neg \exists K' \subset K \mid K' \rightarrow R^+$

FDs and keys

- FDs allow to express integrity constraints that cannot express by super keys
- Example
 - Relation schema *Customer_Loan(cust_name, loan_number, branch_name, amount)*
 - We have
$$\begin{aligned} loan_number &\rightarrow amount \\ loan_number &\rightarrow branch_name \end{aligned}$$
 - But we do not have
 - Many customers can borrow money jointly from a bank
$$loan_number \rightarrow cust_name$$

Apply FDs to find keys

■ Idea

- (1) Find all possible combinations from R^+
- (2) Assume that S contains all combinations $K \subseteq R^+$ satisfying the condition (i) ($K^+ = R^+$)
 - Each combination K is a super key of R
- (3) $\forall K \in S$, if $\exists K' \mid K' \subset K$ then remove K from S

Apply FDs to find keys

- In fact, (2) and (3)
 - From the combination having at least elements
 - If a combination K_i satisfying the conditions (i) is found, we will remove combinations containing K_i right away
- Problem
 - The number of combinations from R^+ can be huge if R^+ is large
- Solution
 - Need to limit the number of combinations

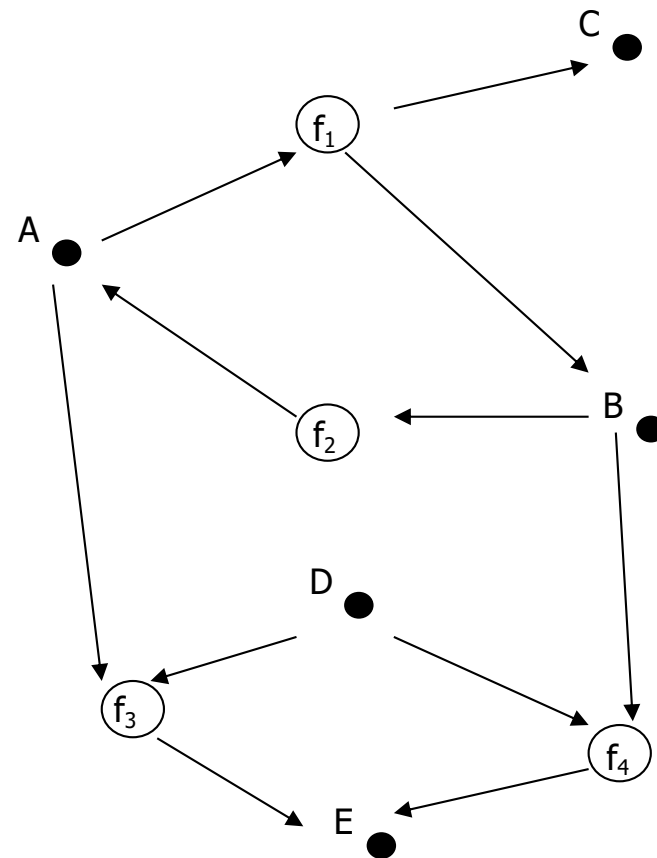
Graph of FDs

- Consists of
 - Node
 - FD : ○ FD_name
 - Attribute : ● Attribute_name
 - Arc
 - From a FD to an attribute or vice versa
 - To represent a FD $f: A \rightarrow B$
 - * An arc starts from the attribute A to the FD f
 - * An arc starts from the FD f to the attribute B
- Whether the graph contains cycles or not?

Example

- Given $R(A,B,C,D,E)$
- And $F = \{$
 $f_1 : A \rightarrow BC$
 $f_2 : B \rightarrow A$
 $f_3 : AD \rightarrow E$
 $f_4 : BD \rightarrow E \}$

- Draw the graph of F



Apply FDs to find keys

■ How to limit

- Source attributes

- A is a source attribute if $\neg \exists f : X \rightarrow Y \in F \mid A \in Y$
- In the graph of FDs, source attributes do not have arcs with heads, have arcs with tails
- All source attributes must exist in every key of the relation R

- Destination attributes

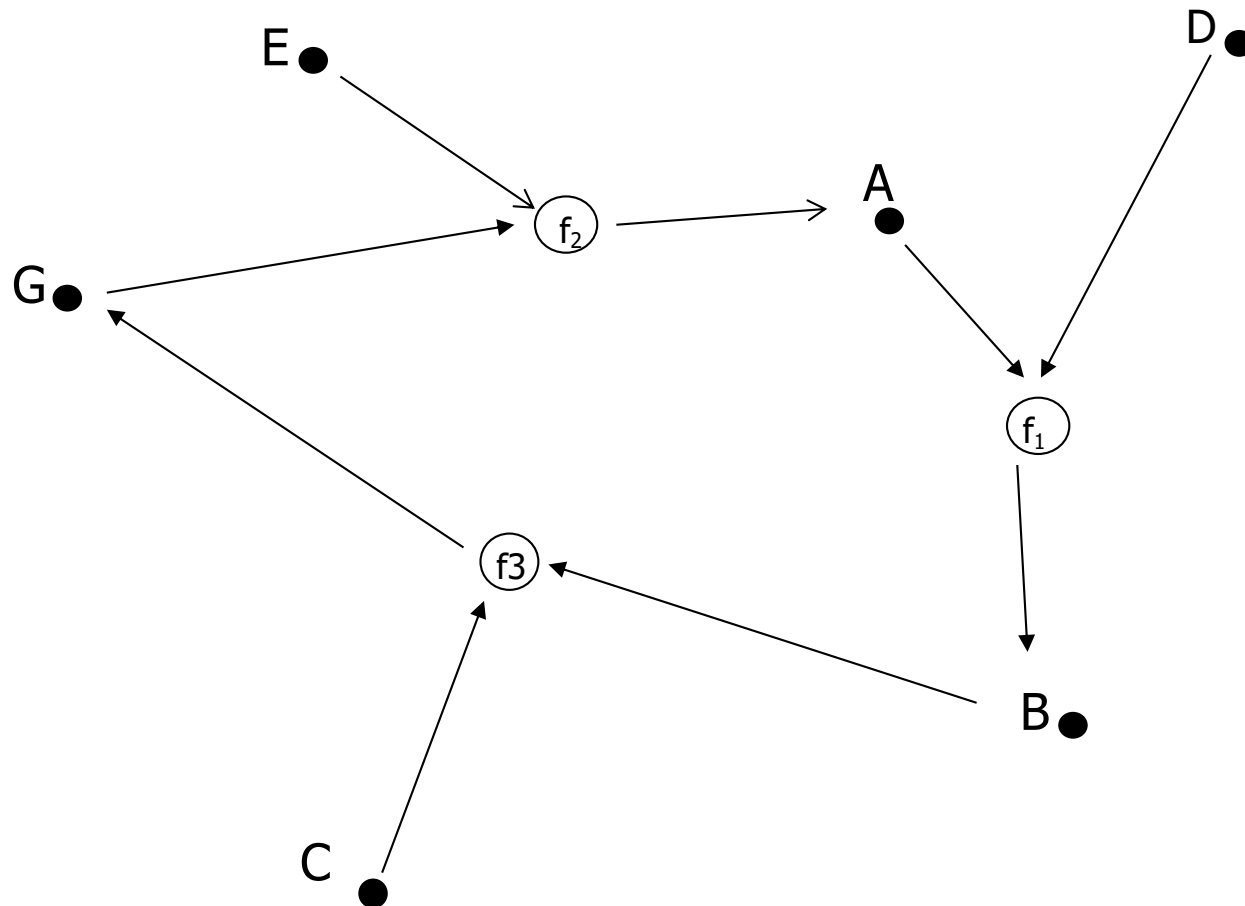
- B is destination attributes if $\neg \exists f : X \rightarrow Y \in F \mid B \in X$
- In the graph of FDs, destination attributes have arcs with heads, do not have arcs with tails
- Destination attributes do not exist in any keys of R

Example

- Given $Q(ABCDEFG)$
- And $F = \{f_1: AD \rightarrow B; f_2: EG \rightarrow A; f_3: BC \rightarrow G\}$
- Find keys of Q

Example

- Graph of F



Example

- The graph has no destination attributes
- Three source attributes : E, D, C
- A cycle via nodes : G, A, B
 - All possible combinations : G, A, B, GA, GB, AB, GAB
- Consider : $K_1 = (\text{EDCG}): K_1 \rightarrow Q^+$
 $K_2 = (\text{EDCA}): K_2 \rightarrow Q^+$
 $K_3 = (\text{EDCB}): K_3 \rightarrow Q^+$
- Three combinations are minimal
- K_1, K_2, K_3 are keys of Q

Content

- Introduction
- Functional dependency
- **Normal forms**
 - Normalization
 - Decomposition
 - The Tableau algorithm
 - 1NF
 - 2NF
 - 3NF
 - BCNF

Remind

- In relational design, 2 approaches
 - (1) Perform a conceptual schema design, then map it into a set of relations
 - (2) Design relations based on external knowledge derived from an existing implementation of files or form of reports
- After this, we have
 - Set of FDs for each relation
 - Each relation has a designated primary key
- Next step
 - Evaluate the relations for goodness
 - Decompose relations to achieve high *normal forms*

Normalization

- Idea

- Take a relation schema through a series of test to “certify” whether it satisfies a certain *normal form*

- Normal form

- The highest normal form condition that a relation meets
- The degree to which a relation has been normalized

Normalization

- A process of analyzing the given relation schemas based on their FDs and PK to achieve properties
 - Minimizing redundancy
 - Minimizing the insertion, deletion, and update anomalies
- Unsatisfactory relation schemas
 - Decomposed into smaller relation schemas

Decomposition

- Decomposing relations
 - Splitting the set of attributes of a relation to make the schemas of two new relations
- Two properties must confirm
 - (1) Lossless join
 - Any instances of the original relation can be found from relation schemas created after decomposition
 - (2) Dependency preservation
 - FDs of the original relation must be preserved in relation schemas created after decomposition

(1) Lossless join

- Review the example of *EMP_PRO*

<u>Ssn</u>	<u>Pnumber</u>	Hours	Ename	Pname	Plocation
123456789	1	32.5	Smith, John B.	ProductX	Bellaire
123456789	2	7.5	Smith, John B.	ProductY	Sugarland
666884444	3	40.0	Narayan, Ramesh K.	ProductZ	Houston
453453453	1	20.0	English, Joyce A.	ProductX	Bellaire
453453453	2	20.0	English, Joyce A.	ProductY	Sugarland
333445555	2	10.0	Wong, Franklin T.	ProductY	Sugarland
333445555	3	10.0	Wong, Franklin T.	ProductZ	Houston
333445555	10	10.0	Wong, Franklin T.	Computerization	Stafford
333445555	20	10.0	Wong, Franklin T.	Reorganization	Houston
999887777	30	30.0	Zelaya, Alicia J.	Newbenefits	Stafford
999887777	10	10.0	Zelaya, Alicia J.	Computerization	Stafford
987987987	10	35.0	Jabbar, Ahmad V.	Computerization	Stafford
987987987	30	5.0	Jabbar, Ahmad V.	Newbenefits	Stafford
987654321	30	20.0	Wallace, Jennifer S.	Newbenefits	Stafford
987654321	20	15.0	Wallace, Jennifer S.	Reorganization	Houston
888665555	20	Null	Borg, James E.	Reorganization	Houston

(1) Lossless join

■ Split *EMP_PRO*

EMP_LOCS

Ename	Plocation
Smith, John B.	Bellaire
Smith, John B.	Sugarland
Narayan, Ramesh K.	Houston
English, Joyce A.	Bellaire
English, Joyce A.	Sugarland
Wong, Franklin T.	Sugarland
Wong, Franklin T.	Houston
Wong, Franklin T.	Stafford
Zelaya, Alicia J.	Stafford
Jabbar, Ahmad V.	Stafford
Wallace, Jennifer S.	Stafford
Wallace, Jennifer S.	Houston
Borg, James E.	Houston

EMP_PROJ1

Ssn	Pnumber	Hours	Pname	Plocation
123456789	1	32.5	ProductX	Bellaire
123456789	2	7.5	ProductY	Sugarland
666884444	3	40.0	ProductZ	Houston
453453453	1	20.0	ProductX	Bellaire
453453453	2	20.0	ProductY	Sugarland
333445555	2	10.0	ProductY	Sugarland
333445555	3	10.0	ProductZ	Houston
333445555	10	10.0	Computerization	Stafford
333445555	20	10.0	Reorganization	Houston
999887777	30	30.0	Newbenefits	Stafford
999887777	10	10.0	Computerization	Stafford
987987987	10	35.0	Computerization	Stafford
987987987	30	5.0	Newbenefits	Stafford
987654321	30	20.0	Newbenefits	Stafford
987654321	20	15.0	Reorganization	Houston
888665555	20	NULL	Reorganization	Houston

(1) Lossless join

- Attempt a natural join operation

not
lossless
join

	Ssn	Pnumber	Hours	Pname	Plocation	Ename
	123456789	1	32.5	ProductX	Bellaire	Smith, John B.
*	123456789	1	32.5	ProductX	Bellaire	English, Joyce A.
	123456789	2	7.5	ProductY	Sugarland	Smith, John B.
*	123456789	2	7.5	ProductY	Sugarland	English, Joyce A.
*	123456789	2	7.5	ProductY	Sugarland	Wong, Franklin T.
	666884444	3	40.0	ProductZ	Houston	Narayan, Ramesh K.
*	666884444	3	40.0	ProductZ	Houston	Wong, Franklin T.
*	453453453	1	20.0	ProductX	Bellaire	Smith, John B.
	453453453	1	20.0	ProductX	Bellaire	English, Joyce A.
*	453453453	2	20.0	ProductY	Sugarland	Smith, John B.
	453453453	2	20.0	ProductY	Sugarland	English, Joyce A.
*	453453453	2	20.0	ProductY	Sugarland	Wong, Franklin T.
*	333445555	2	10.0	ProductY	Sugarland	Smith, John B.
*	333445555	2	10.0	ProductY	Sugarland	English, Joyce A.
	333445555	2	10.0	ProductY	Sugarland	Wong, Franklin T.
*	333445555	3	10.0	ProductZ	Houston	Narayan, Ramesh K.
	333445555	3	10.0	ProductZ	Houston	Wong, Franklin T.
	333445555	10	10.0	Computerization	Stafford	Wong, Franklin T.
*	333445555	20	10.0	Reorganization	Houston	Narayan, Ramesh K.
	333445555	20	10.0	Reorganization	Houston	Wong, Franklin T.

*
*
*

(1) Lossless join

■ Definition

- A decomposition $D = \{R_1, R_2, \dots, R_m\}$ of R
- Has the lossless join property with respect to the set of dependencies F on R
- If for every relation state r that satisfies F , the followings holds

$$*(\pi_{R_1}(r), \dots, \pi_{R_m}(r)) = r$$

- Where $*$ is the natural join of all relations in D

(1) Lossless join

■ Example: Decomposition 1

<u>SSN</u>	<u>PNUM</u>	HOURS	ENAME	PNAME	PLOCATION
------------	-------------	-------	-------	-------	-----------

$F = \{ \text{SSN} \rightarrow \text{ENAME},$
 $\text{PNUM} \rightarrow \text{PNAME}, \text{PLOCATION},$
 $\text{SSN}, \text{PNUM} \rightarrow \text{HOURS} \}$



R1

SSN	ENAME
-----	-------

R2

PNUM	PNAME	PLOCATION
------	-------	-----------

R3

SSN	PNUM	HOURS
-----	------	-------

lossless join

(1) Lossless join

■ Example: Decomposition 2

<u>SSN</u>	<u>PNUM</u>	HOURS	ENAME	PNAME	PLOCATION
------------	-------------	-------	-------	-------	-----------

$F = \{ \text{SSN} \rightarrow \text{ENAME},$
 $\text{PNUM} \rightarrow \text{PNAME}, \text{PLOCATION},$
 $\text{SSN}, \text{PNUM} \rightarrow \text{HOURS} \}$



R1

ENAME	PLOCATION
-------	-----------

not lossless join

R2

SSN	PNUM	HOURS	PNAME	PLOCATION
-----	------	-------	-------	-----------

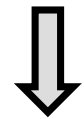
The Tableau – Example 1

	A1 SSN	A2 ENAME	A3 PNUM	A4 PNAME	A5 PLOCATION	A6 HOURS
R1	b11	b12	b13	b14	b15	b16
R2	b21	b22	b23	b24	b25	b26
R3	b31	b32	b33	b34	b35	b36



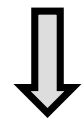
R1	a1	a2	b13	b14	b15	b16
R2	b21	b22	a3	a4	a5	b26
R3	a1	b32	a3	b34	b35	a6

The Tableau – Example 1



SSN → ENAME

	SSN	ENAME				
R1	a1	a2	b13	b14	b15	b16
R2	b21	b22	a3	a4	a5	b26
R3	a1	a2	a3	b34	b35	a6



PNUM → PNAME, PLOCATION

	PNUM	PNAME	PLOCATION			
R1	a1	a2	b13	b14	b15	b16
R2	b21	b22	a3	a4	a5	b26
R3	a1	a2	a3	a4	a5	a6

The Tableau – Example 1



SSN, PNUM $\rightarrow$ HOURS

	SSN		PNUM			HOURS
R1	a1	a2	b13	b14	b15	b16
R2	b21	b22	a3	a4	a5	b26
R3	a1	a2	a3	a4	a5	a6

The matrix is not changed

The Tableau – Example 1

EMP_PRO

a1	a2	b13	b14	b15	b16
b21	b22	a3	a4	a5	b26
a1	a2	a3	a4	a5	a6

R1

a1	a2
b21	b22

R2

b13	b14	b15
a3	a4	a5

R3

a1	b13	b16
b21	a3	b26
a1	a3	a6

The Tableau – Example 1

- $R1 * R3 = R13$

$$R1 \begin{pmatrix} a1 & a2 \\ b21 & b22 \end{pmatrix} \quad R3 \begin{pmatrix} a1 & b13 & b16 \\ b21 & a3 & b26 \\ a1 & a3 & a6 \end{pmatrix}$$

$$R13 \begin{pmatrix} a1 & a2 & b13 & b16 \\ a1 & a2 & a3 & a6 \\ b21 & b22 & a3 & b26 \end{pmatrix}$$

The Tableau – Example 1

- $R13 * R2 = EMP_PRO$

R2

(

b13

b14

b15

a3

a4

a5

)

R13

(

a1

a2

b13

b16

a1

a2

a3

a6

b21

b22

a3

b26

)

EMP_PRO

(

a1

a2

b13

b14

b15

b16

b21

b22

a3

a4

a5

b26

a1

a2

a3

a4

a5

a6

)

The Tableau – Example 2

	A1 SSN	A2 ENAME	A3 PNUM	A4 PNAME	A5 PLOCATION	A6 HOURS
R1	b11	b12	b13	b14	b15	b16
R2	b21	b22	b23	b24	b25	b26



R1	b11	a2	b13	b14	a5	b16
R2	a1	b22	a3	a4	a5	a6



SSN → ENAME

PNUM → PNAME, PLOCATION

SSN, PNUM → HOURS

The matrix is not changed

The Tableau – Example 2

EMP_PRO

R1	b11	a2	b13	b14	a5	b16
R2	a1	b22	a3	a4	a5	a6

R1	a2	a5
	b22	a5

R2	b11	b13	b14	a5	b16
	a1	a3	a4	a5	a6

The Tableau – Example 2

- $R1 * R2 \neq EMP_PRO$

R1

a2

a5

b22

a5

R2

b11

b13

b14

a5

b16

a1

a3

a4

a5

a6

b11

a2

b13

b14

a5

b16

a1

a2

a3

a4

a5

a6

b11

b22

b13

b14

a5

b16

a1

b22

a3

a4

a5

a6

Spurious tuples

(2) Dependency preservation

- Why do we want to keep dependencies?
- Losing a dependency
 - The corresponding constraint can be check only through natural join of the appropriate relations in the composition → very expensive

(2) Dependency preservation

■ Example

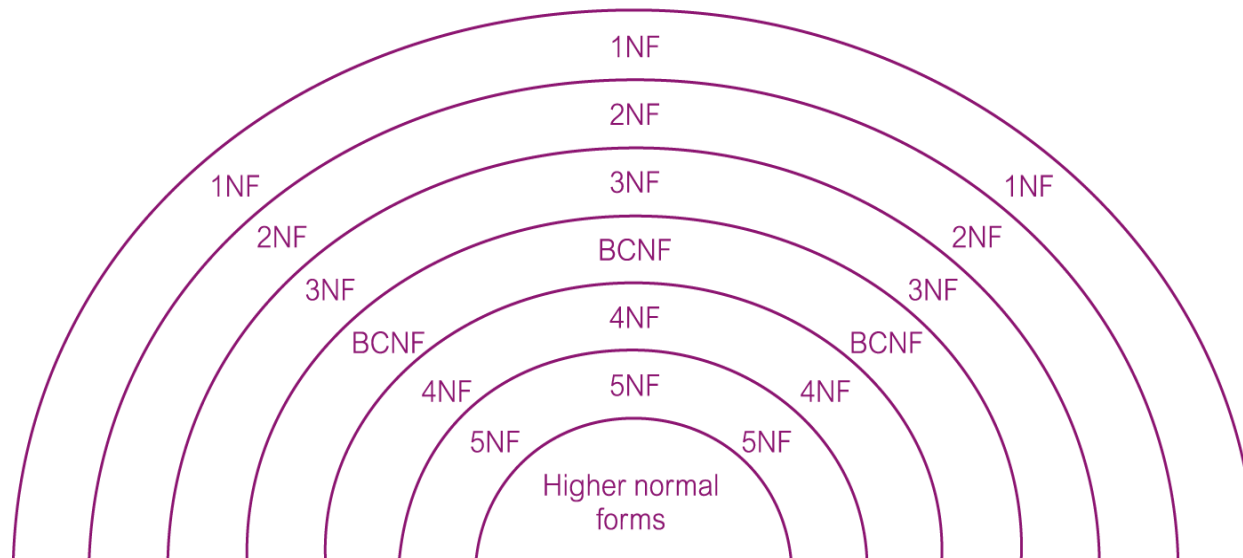
- $R(A, B, C); F = \{A \rightarrow B, B \rightarrow C\}$
- Decomposition of R : $R_1(A, C)$ and $R_2(B, C)$
- Does this composition preserve F ?
- In R_1 , the following dependencies hold
 - $F_1 = \{A \rightarrow A, C \rightarrow C, A \rightarrow C, AC \rightarrow AC\}$
- In R_2 , the following dependencies hold
 - $F_2 = \{B \rightarrow B, C \rightarrow C, B \rightarrow C, BC \rightarrow BC\}$
- Nontrivial dependencies hold on R_1 & R_2
 - $F' \equiv (F_1 \cup F_2) = \{A \rightarrow C, B \rightarrow C\}$ not dependency preserving

(2) Dependency preservation

■ Example

- $R(A, B, C); F = \{A \rightarrow B, B \rightarrow C\}$
- Decomposition of R : $R_1(A, B)$ and $R_2(B, C)$
- Does this composition preserve F ?
- In R_1 , the following dependencies hold
 - $F_1 = \{A \rightarrow A, C \rightarrow C, A \rightarrow B, AB \rightarrow AB\}$
- In R_2 , the following dependencies hold
 - $F_2 = \{B \rightarrow B, C \rightarrow C, B \rightarrow C, BC \rightarrow BC\}$
- Nontrivial dependencies hold on R_1 & R_2
 - $F' \equiv (F_1 \cup F_2) = \{A \rightarrow B, B \rightarrow C\}$ **dependency preserving**

Normal forms



Example

■ Schema List_of_Classes

ClassID	No. Stu	StuName_1	Grade_11	...	Grade_15	StuName_2	Grade_21	...	Grade_25	...
11A2	30	N V A	7	...	8	T T C	8	...	8	... 30 times
12A1	28	V V B	10	...	6	P V D	6	...	6	... 28 times

5 times

Anomaly

(cram too much into a single relation – “relations within relations”)

Example

■ Schema Customer_Rent_Rooms

CustID	CustName	RoomID	RoomAddr	CheckIn	CheckOut	Price	Owner ID	Owner Name
CR76	N V A	PG04	6 NVC Q5	1/6/03	31/8/04	350	CO40	X
		PG16	5 NT Q10	1/9/04	1/9/05	450	CO93	Y
CR56	P T B	PG06	6 NVC Q5	1/9/02	10/6/03	350	CO40	X
		PG36	2 NTMK Q3	10/10/03	1/12/04	375	CO93	Y
		PG16	5 NT Q10	1/11/05	10/8/06	450	CO93	Y

Anomaly

(Multivalued attributes – “relations as attribute values within tuples”)

Anomaly → 1NF

- Disallows
 - “relations within relations”
 - “relations as attribute values within tuples”
- The only attribute values permitted are single atomic values

Anomaly → 1NF

■ Decompose relations

ClassID	No. Stu	StuName 1	Grade_ 11	...	Grade_ 15	StuName 2	Grade_ 21	...	Grade_ 25	...
11A2	30	N V A	7	...	8	T T C	8	...	8	...
12A1	28	V V B	10	...	6	P V D	6	...	6	...

List_of_students

ClassID	No. Stu	StuName
11A2	30	N V A
11A2	30	T T C
12A1	28	V V B
12A1	28	P V D

Grade_Student_Class

ClassID	StuName	Course	Grade
11A2	N V A	Literature	7
11A2	N V A	Maths	7
11A2	T T C	Literature	8
11A2	T T C	Maths	7
12A1	V V C	Maths	10
12A1	P V D	Physics	6

Anomaly → 1NF

- Change multivalued attributes to atomic attributes

CustID	CustName	RoomID	RoomAddr	CheckIn	CheckOut	Price	Owner ID	Owner Name
CR76	N V A	PG04	6 NVC Q5	1/6/03	31/8/04	350	CO40	X
		PG16	5 NT Q10	1/9/04	1/9/05	450	CO93	Y
CR56	P T B	PG06	6 NVC Q5	1/9/02	10/6/03	350	CO40	X
		PG36	2 NTMK Q3	10/10/03	1/12/04	375	CO93	Y
		PG16	5 NT Q10	1/11/05	10/8/06	450	CO93	Y

CustID	CustName	RoomID	RoomAddr	CheckIn	CheckOut	Price	Owner ID	Owner Name
CR76	N V A	PG04	6 NVC Q5	1/6/03	31/8/04	350	CO40	X
CR76	N V A	PG16	5 NT Q10	1/9/04	1/9/05	450	CO93	Y
CR56	P T B	PG06	6 NVC Q5	1/9/02	10/6/03	350	CO40	X
CR56	P T B	PG36	2 NTMK Q3	10/10/03	1/12/04	375	CO93	Y
CR56	P T B	PG16	5 NT Q10	1/11/05	10/8/06	450	CO93	Y

Anomaly → 1NF

■ Decompose relations

CustID	CustName	RoomID	RoomAddr	CheckIn	CheckOut	Price	Owner ID	Owner Name
CR76	N V A	PG04	6 NVC Q5	1/6/03	31/8/04	350	CO40	X
		PG16	5 NT Q10	1/9/04	1/9/05	450	CO93	Y
CR56	P T B	PG06	6 NVC Q5	1/9/02	10/6/03	350	CO40	X
		PG36	2 NTMK Q3	10/10/03	1/12/04	375	CO93	Y
		PG16	5 NT Q10	1/11/05	10/8/06	450	CO93	Y

Customer

CustID	CustName
CR76	N V A
CR56	P T B

Rent_Room

CustID	RoomID	RoomAddr	CheckIn	CheckOut	Price	Owner ID	Owner Name
CR76	PG04	6 NVC Q5	1/6/03	31/8/04	350	CO40	X
CR76	PG16	5 NT Q10	1/9/04	1/9/05	450	CO93	Y
CR56	PG06	6 NVC Q5	1/9/02	10/6/03	350	CO40	X
CR56	PG36	2 NTMK Q3	10/10/03	1/12/04	375	CO93	Y
CR56	PG16	5 NT Q10	1/11/05	10/8/06	450	CO93	Y

1NF

Discussion

- Redundancy still exists

Rent_Room

CustID	RoomID	RoomAddr	CheckIn	CheckOut	Price	OwnerID	OwnerName
CR76	PG04	6 NVC Q5	1/6/03	31/8/04	350	CO40	X
CR76	PG16	5 NT Q10	1/9/04	1/9/05	450	CO93	Y
CR56	PG04	6 NVC Q5	1/9/02	10/6/03	350	CO40	X
CR56	PG36	2 NTMK Q3	10/10/03	1/12/04	375	CO93	Y
CR56	PG16	5 NT Q10	1/11/05	10/8/06	450	CO93	Y

1NF → 2NF

- Remove incomplete FDs
- A relation R is in 2NF
 - Satisfies 1NF
 - Nonprime attributes in R is ***completely functionally dependent*** on the key of R

1NF → 2NF

- Schema **Rent_Room**

- Key: CustID, RoomID

**CustID, RoomID → RoomAddr, CheckIn, CheckOut,
Price, OwnerID, OwnerName**

- Have a following FD

RoomID → RoomAddr, Price, OwnerID, OwnerName

- This FD is incomplete one on keys

1NF → 2NF

- Decompose schema **Rent_Room**

Room	RoomID	RoomAddr	Price	Owner ID	Owner Name
	PG04	6 NVC Q5	350	CO40	X
	PG16	5 NT Q10	450	CO93	Y
	PG36	2 NTMK Q3	375	CO93	Y

Rent	CustID	RoomID	CheckIn	CheckOut
	CR76	PG04	1/6/03	31/8/04
	CR76	PG16	1/9/04	1/9/05
	CR56	PG04	1/9/02	10/6/03
	CR56	PG36	10/10/03	1/12/04
	CR56	PG16	1/11/05	10/8/06

1NF → 2NF

Customer

CustID	CustName
CR76	N V A
CR56	P T B

1NF

Rent_Room

CustID	RoomID	RoomAddr	CheckIn	CheckOut	Price	Owner ID	Owner Name
CR76	PG04	6 NVC Q5	1/6/03	31/8/04	350	CO40	X
CR76	PG16	5 NT Q10	1/9/04	1/9/05	450	CO93	Y
CR56	PG06	6 NVC Q5	1/9/02	10/6/03	350	CO40	X
CR56	PG36	2 NTMK Q3	10/10/03	1/12/04	375	CO93	Y
CR56	PG16	5 NT Q10	1/11/05	10/8/06	450	CO93	Y

Rent

CustID	RoomID	CheckIn	CheckOut
CR76	PG04	1/6/03	31/8/04
CR76	PG16	1/9/04	1/9/05
CR56	PG04	1/9/02	10/6/03
CR56	PG36	10/10/03	1/12/04
CR56	PG16	1/11/05	10/8/06

Room

RoomID	RoomAddr	Price	Owner ID	Owner Name
PG04	6 NVC Q5	350	CO40	X
PG16	5 NT Q10	450	CO93	Y
PG36	2 NTMK Q3	375	CO93	Y

2NF

Customer

CustID	CustName
CR76	N V A
CR56	P T B

Discussion

- Redundancy still exists

Room

RoomID	RoomAddr	Price	Owner ID	Owner Name
PG04	6 NVC Q5	350	CO40	X
PG16	5 NT Q10	450	CO93	Y
PG36	2 NTMK Q3	375	CO93	Y

2NF → 3NF

- Remove transitive FDs
- A relation R is in 3NF
 - Satisfies 2NF
 - Nonprime attributes in R are ***not transitive dependent*** on the key of R

2NF → 3NF

■ Schema Room

- Key: RoomID

RoomID → RoomAddress, Price, OwnerID, OwnerName

RoomID → OwnerID

RoomID → OwnerName

(FD6)

- Have one more FD

OwnerID → OwnerName

- This FD is a transitive FD on keys

2NF → 3NF

■ Decompose Room

Room

RoomID	RoomAddr	Price	OwnerID
PG04	6 NVC Q5	350	CO40
PG16	5 NT Q10	450	CO93
PG36	2 NTMK Q3	375	CO93

Owner

OwnerID	OwnerName
CO40	X
CO93	Y

2NF → 3NF

Rent

CustID	RoomID	CheckIn	CheckOut
CR76	PG04	1/6/03	31/8/04
CR76	PG16	1/9/04	1/9/05
CR56	PG04	1/9/02	10/6/03
CR56	PG36	10/10/03	1/12/04
CR56	PG16	1/11/05	10/8/06

Room

RoomID	RoomAddr	Price	Owner ID	Owner Name
PG04	6 NVC Q5	350	CO40	X
PG16	5 NT Q10	450	CO93	Y
PG36	2 NTMK Q3	375	CO93	Y

2NF

Customer

CustID	CustName
CR76	N V A
CR56	P T B

Rent

CustID	RoomID	CheckIn	CheckOut
CR76	PG04	1/6/03	31/8/04
CR76	PG16	1/9/04	1/9/05
CR56	PG04	1/9/02	10/6/03
CR56	PG36	10/10/03	1/12/04
CR56	PG16	1/11/05	10/8/06

Room

3NF

RoomID	RoomAddr	Price	OwnerID
PG04	6 NVC Q5	350	CO40
PG16	5 NT Q10	450	CO93
PG36	2 NTMK Q3	375	CO93

Customer

CustID	CustName
CR76	N V A
CR56	P T B

Owner

OwnerID	OwnerName
CO40	X
CO93	Y

Discussion

- Redundancy still exists when

Rent

CustID	RoomID	CheckIn	CheckOut	EmplID
CR76	PG04	1/6/03	31/8/04	SG37
CR76	PG16	1/9/04	1/9/05	SG24
CR56	PG04	1/9/02	10/6/03	SG37
CR56	PG36	10/10/03	1/12/04	SG05

EmplID → RoomID

3NF → BCNF

- Boyce-Codd normal form
- Remove FDs whose left-hand side does not contain key attributes
- A relation R is in BCNF
 - Satisfies 3NF
 - The left-hand side of nontrivial FDs must contain the super key of R

3NF → BCNF

■ Schema Rent

- Key: CustID, RoomID

CustID, RoomID → CheckIn, CheckOut, EmplID

- Have a FD whose left-hand side does not contain key attributes

EmplID → RoomID

3NF → BCNF

- Decompose the schema **Rent**
 - Specify keys again
 - Relation Empl-Room : EmplID, RoomID
 - Relation Rent : CustID, EmplID

Empl-Room

EmplID	RoomID
SG37	PG04
SG24	PG16
SG05	PG36

Rent

CustID	EmplID	CheckIn	CheckOut
CR76	SG37	1/6/03	31/8/04
CR76	SG24	1/9/04	1/9/05
CR56	SG37	1/9/02	10/6/03
CR56	SG05	10/10/03	1/12/04

3NF → BCNF

Rent	CustID	RoomID	CheckIn	CheckOut	EmplID
	CR76	PG04	1/6/03	31/8/04	SG37
	CR76	PG16	1/9/04	1/9/05	SG24
	CR56	PG04	1/9/02	10/6/03	SG37
	CR56	PG36	10/10/03	1/12/04	SG05

Empl-Room

<u>EmplID</u>	RoomID
SG37	PG04
SG24	PG16
SG05	PG36

Rent

<u>CustID</u>	<u>EmplID</u>	CheckIn	CheckOut
CR76	SG37	1/6/03	31/8/04
CR76	SG24	1/9/04	1/9/05
CR56	SG37	1/9/02	10/6/03
CR56	SG05	10/10/03	1/12/04

- Why did we choose this decomposition?

3NF → BCNF

Rent

CustID	RoomID	CheckIn	CheckOut	EmplID
CR76	PG04	1/6/03	31/8/04	SG37
CR76	PG16	1/9/04	1/9/05	SG24
CR56	PG04	1/9/02	10/6/03	SG37
CR56	PG36	10/10/03	1/12/04	SG05

3NF

Room

RoomID	RoomAddr	Price	OwnerID
PG04	6 NVC Q5	350	CO40
PG16	5 NT Q10	450	CO93
PG36	2 NTMK Q3	375	CO93

Customer

CustID	CustName	OwnerID	OwnerName
CR76	N V A	CO40	X
CR56	P T B	CO93	Y

Owner

Rent

CustID	EmplID	CheckIn	CheckOut
CR76	SG37	1/6/03	31/8/04
CR76	SG24	1/9/04	1/9/05
CR56	SG37	1/9/02	10/6/03
CR56	SG05	10/10/03	1/12/04

BCNF

Room

RoomID	RoomAddr	Price	OwnerID	Owner ID	Owner Name
PG04	6 NVC Q5	350	CO40		
PG16	5 NT Q10	450	CO93		
PG36	2 NTMK Q3	375	CO93	CO40	X
				CO93	Y

Empl-Room

EmplID	RoomID
SG37	PG04
SG24	PG16
SG05	PG36

Customer

CustID	CustName
CR76	N V A
CR56	P T B

Content

- Introduction
- Functional dependency
 - 1NF
 - 2NF
 - 3NF
 - BCNF
- Multivalued dependency
 - 4NF

Discussion

- A kind of redundancy that is not related to FDs

RoomID
↓

Rent

CustID	EmplID	CheckIn	CheckOut
CR76	SG37	1/6/03	31/8/04
CR76	SG37	1/9/04	1/9/05
CR76	SG24	1/6/03	31/8/04
CR76	SG24	1/9/04	1/9/05
CR56	SG37	1/9/02	10/6/03
CR56	SG05	10/10/03	1/12/04
...	...	...	...

Multivalued dependency

■ Schema Rent

- Key : CustID, EmplID, CheckIn, CheckOut
- FD : none
- MVD

CustID → EmplID

CustID → CheckIn, CheckOut

Discussion

- A kind of redundancy that is not related to FDs

Rent



CustID	EmplID	CheckIn	CheckOut	Service	Interest
CR76	SG37	1/6/03	31/8/04	cleaning	communication
CR76	SG37	1/6/03	31/8/04	making beds	lifestyles
CR76	SG24	1/9/04	1/9/05	laundry	communication
CR76	SG24	1/9/04	1/9/05	replacing towels	lifestyles
CR56	SG37	1/9/02	10/6/03	laundry	cleanliness
CR56	SG05	10/10/03	1/12/04	replacing towels	cleanliness
CR76	SG37	1/6/03	31/8/04	cleaning	lifestyles
...	...	...	...	...	...

Multivalued dependency

■ Schema Rent

- FD

CustID, EmplID → CheckIn, CheckOut

- MVD

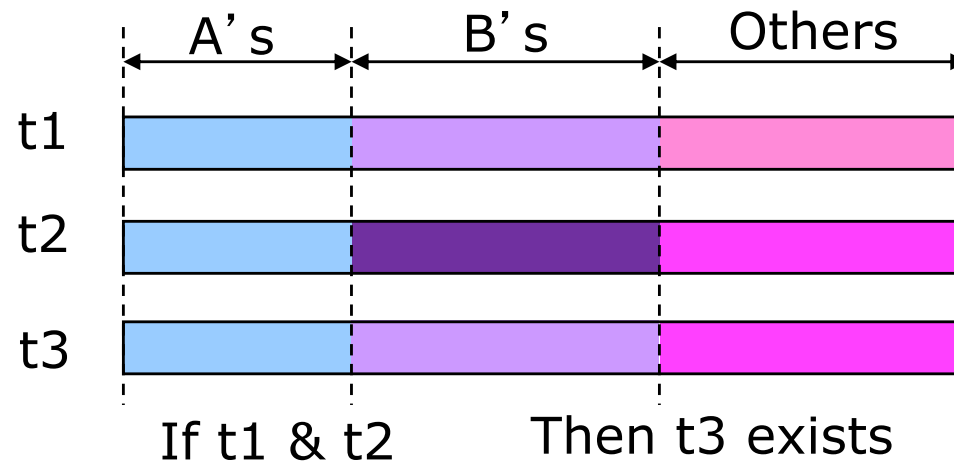
CustID, EmplID, CheckIn, CheckOut → Service

CustID, EmplID, CheckIn, CheckOut → Interest

Multivalued dependency

■ Definition

- A relation R , fix values for one set of attributes
- Values in certain other attributes are independent of values of all other attributes in R



■ Denotation

$$\mathbf{A_1, A_2, \dots, A_n \twoheadrightarrow B_1, B_2, \dots, B_m}$$

$A_1, A_2, \dots, A_n$ multi-determine $B_1, B_2, \dots, B_m$

Rules

- Trivial multivalued dependency

$$A_1, A_2, \dots, A_n \twoheadrightarrow B_1, B_2, \dots, B_m$$

$$\left[\begin{array}{l} B_1, B_2, \dots, B_m \subseteq A_1, A_2, \dots, A_n \\ B_1, B_2, \dots, B_m \cup A_1, A_2, \dots, A_n = \text{all attributes} \end{array} \right.$$

- FD is a MVD rule If $X \rightarrow Y$ Then $X \twoheadrightarrow Y$

- Intersection rule If $X \twoheadrightarrow Y$ and $X \twoheadrightarrow Z$ Then $X \twoheadrightarrow Y \cap Z$

- Transitive rule If $X \twoheadrightarrow Y$ and $Y \twoheadrightarrow Z$ Then $X \twoheadrightarrow Z - Y$

BCNF $\rightarrow$ 4NF

- Remove MVDs
- A relation R with MVDs is in 4NF
 - For each non-trivial $X \twoheadrightarrow Y$
 - X is a key

BCNF → 4NF

- Decompose the schema **Rent**

Rent	CustID	EmplID	CheckIn	CheckOut
	CR76	SG37	1/6/03	31/8/04
	CR76	SG37	1/9/04	1/9/05
	CR76	SG24	1/6/03	31/8/04
	CR76	SG24	1/9/04	1/9/05
	CR56	SG37	1/9/02	10/6/03
	CR56	SG05	10/10/03	1/12/04

Rent1	CustID	EmplID
	CR76	SG37
	CR76	SG24
	CR56	SG37
	CR56	SG05

Rent2	CustID	CheckIn	CheckOut
	CR76	1/6/03	31/8/04
	CR76	1/9/04	1/9/05
	CR56	1/9/02	10/6/03
	CR56	10/10/04	1/12/04

4NF decomposition

- CustID, EmplID, CheckIn, CheckOut → Service

Rent	CustID	EmplID	CheckIn	CheckOut	Service	Interest
	CR76	SG37	1/6/03	31/8/04	cleaning	communication
	CR76	SG37	1/6/03	31/8/04	making beds	lifestyles
	CR76	SG24	1/9/04	1/9/05	laundry	communication
	CR76	SG24	1/9/04	1/9/05	replacing towels	lifestyles
	CR56	SG37	1/9/02	10/6/03	laundry	cleanliness
	CR56	SG05	10/10/03	1/12/04	replacing towels	cleanliness
	CR76	SG37	1/6/03	31/8/04	cleaning	lifestyles

- Rent1(CustID, EmplID, CheckIn, CheckOut, Service)
- Rent2 (CustID, EmplID, CheckIn, CheckOut, Interest)

4NF decomposition

- **CustID, EmplID → CheckIn, CheckOut**
- Rent1(CustID, EmplID, CheckIn, CheckOut, Service)
- Rent2(CustID, EmplID, CheckIn, CheckOut, Interest)
- Rent3(CustID, EmplID, CheckIn, CheckOut)
- Rent4(CustID, EmplID, Service)

4NF decomposition

- **CustID, EmplID → CheckIn, CheckOut**
- Rent1(CustID, EmplID, CheckIn, CheckOut, Service)
- Rent2(CustID, EmplID, CheckIn, CheckOut, Interest)
- Rent3(CustID, EmplID, CheckIn, CheckOut)
- Rent4(CustID, EmplID, Service)
- Rent5(CustID, EmplID, Interest)

Conclusion

- The higher normal form is,
 - The less data redundancy is
 - The slower speed of retrieving data is
- Compromise
 - Data storage
 - Data retrieval speed
- In reality
 - Denormalization

